

Unit -3

6(a) Discuss the various storage allocation strategies in detail.

✓ Storage Allocation Strategies in Compiler Design

Storage allocation is used to assign memory to different variables, data, and functions during program execution. There are **three main strategies**:

1. Static Allocation

- Memory is assigned **at compile time**.
- Size and location of variables are **fixed** before execution.
- Used for **global variables, static variables, and constants**.
- No memory is created or freed during runtime.

Example:

c

CopyEdit

```
int x = 10; // allocated statically
```

Advantages:

- Fast access (no overhead).
- Simple to implement.

Disadvantages:

- Fixed memory size (not flexible).
-

2. Stack Allocation

- Memory is assigned in the form of a **stack** (LIFO).
- Used for **local variables and function calls**.
- When a function is called, a **stack frame** is created.
- Stack memory is **freed automatically** when the function ends.

Example:

c

CopyEdit

```
void fun() {  
    int a = 5; // stack allocated  
}
```

Advantages:

- Efficient memory use.
- Supports recursion.

Disadvantages:

- Limited size (stack overflow possible).

3. Heap Allocation

- Memory is assigned **at runtime** (dynamic memory).
- Used when the size is not known during compilation.
- Managed using **malloc()**, **free()**, **new**, **delete**, etc.

Example:

c

CopyEdit

```
int *p = (int*)malloc(sizeof(int)); // heap allocated
```

Advantages:

- Flexible and allows dynamic memory.

Disadvantages:

- Slower than stack.
- Needs manual memory management (risk of memory leaks).

Summary Table

Strategy	Time of Allocation	Usage	Example
Static Allocation	Compile Time	Global/Static Variables	int x = 10;
Stack Allocation	Runtime (LIFO)	Local Variables, Functions	void fun()
Heap Allocation	Runtime (Dynamic)	Dynamic Data Structures	malloc()

Let me know if you want a **short version** for revision or diagram too.

You said:

Dynamic storage allocation

ChatGPT said:

Here is a **simple and clear answer** for:

✓ Dynamic Storage Allocation

📌 Definition:

Dynamic storage allocation is the process of assigning memory to variables **at runtime**, when the size and number of variables are **not known at compile time**.

It is used to **efficiently use memory** for dynamic data structures like:

- Linked lists
 - Trees
 - Graphs
 - Queues
 - Stacks (implemented using pointers)
-

📦 How It Works:

- Memory is taken from the **heap** section of memory.
 - Functions used for dynamic allocation:
 - In **C**: malloc(), calloc(), realloc(), free()
 - In **C++**: new, delete
-

🧠 Example in C:

c

CopyEdit

```
int *ptr;
```

```
ptr = (int*) malloc(sizeof(int)); // allocates memory at runtime
```

```
*ptr = 10;
```

```
free(ptr); // releases memory
```

✓ Advantages:

- Efficient use of memory.
 - Useful for variable-sized data structures.
 - Allocates memory only when needed.
-

✗ Disadvantages:

- Slower than static or stack allocation.
 - Requires **manual memory management**.
 - Risk of **memory leaks** and **dangling pointers**.
-

🔄 Summary:

Feature	Description
When	At runtime
Memory Source	Heap
Flexibility	High (good for dynamic data structures)
Risk	Memory leaks, if not managed properly

Let me know if you want this in **short points or tabular form** for revision!

You said:

Error detection and recovery

ChatGPT said:

Here is a **simple and exam-ready answer** for:

Error Detection and Recovery in Compiler Design

Error Detection:

It is the process of finding **errors in the source code** during compilation.

Errors are detected in different phases:

- **Lexical Errors** – Invalid tokens (e.g., int 9var;)
 - **Syntax Errors** – Wrong grammar (e.g., missing ; or {})
 - **Semantic Errors** – Meaningless statements (e.g., type mismatch)
 - **Logical Errors** – Program runs but gives wrong result (not caught by compiler)
-

Error Recovery:

After detecting an error, the compiler tries to **recover** and continue checking the rest of the code instead of stopping.

There are **four common error recovery methods**:

1. Panic Mode Recovery

- Skip tokens until a valid statement is found.
 - Simple and fast.
 - Example: Skip until ; or } is found.
-

2. Phrase Level Recovery

- Replace or insert a few tokens to fix small errors.
 - Example: Missing) is inserted automatically.
-

3. Error Productions

- Grammar is modified to include common mistakes.
 - Helps detect specific known errors.
-

4. Global Correction

- Try to make **minimal changes** to fix the program.
 - Expensive and complex, rarely used.
-

Summary Table:

Method	Description
Panic Mode	Skip input until a safe point is found
Phrase Level	Modify tokens to recover locally

Method	Description
--------	-------------

Error Productions	Add special rules for common errors
-------------------	-------------------------------------

Global Correction	Find minimal fix (costly)
-------------------	---------------------------

Let me know if you want a **diagram or quick revision note!**

You said:

Polymorphic functions

ChatGPT said:

Here is a **simple and exam-friendly answer** for:

 Polymorphic Functions

 Definition:

Polymorphic functions are functions that can work with **values of different types**. They are a part of **polymorphism**, which means "**many forms**".

 Types of Polymorphism:

1. **Compile-Time Polymorphism** (Static)
 - Function Overloading
 - Operator Overloading
2. **Run-Time Polymorphism** (Dynamic)
 - Function Overriding using Inheritance

 Examples:

 1. Function Overloading (Compile-Time):

cpp

CopyEdit

```
int add(int a, int b) {  
    return a + b;  
}
```

```
float add(float a, float b) {  
    return a + b;  
}
```

 Same function name add works with **int** and **float** types.

 2. Function Overriding (Run-Time):

cpp

CopyEdit

```
class Animal {  
public:  
    void sound() {  
        cout << "Animal sound";  
    }  
}
```

```
};
```

```
class Dog : public Animal {
```

```
public:
```

```
    void sound() {
```

```
        cout << "Dog barks";
```

```
    }
```

```
};
```

👉 sound() function works **differently** depending on the object (Animal or Dog).

✅ Advantages:

- Code reusability
 - Easy to understand and maintain
 - Supports flexible programming
-

🔄 Summary:

Type	Example	Time
Function Overloading	Same name, different parameters	Compile Time
Function Overriding	Redefine function in derived class	Run Time

Let me know if you want **diagrams or one-line definitions** for last-minute revision!

You said:

(b) What are the possible error recovery actions in lexical analyzer?

ChatGPT said:

Here is a **simple and exam-oriented answer** for:

✅ (b) What are the possible error recovery actions in Lexical Analyzer?

🔍 Lexical Errors:

These errors occur during **lexical analysis**, when the source code contains **invalid tokens**.

Example:

- Misspelled keywords: `intt x = 5;` (should be `int`)
 - Invalid characters: `@x = 10;`
-

🔄 Error Recovery Actions in Lexical Analyzer:

Here are the **common recovery methods** used:

📌 1. Panic Mode Recovery

- Skip characters until a valid token or delimiter (like space, `;`, `{}`) is found.
 - Simple and fast.
-

📌 2. Delete Characters

- Remove the **incorrect character** to make the token valid.
 - Example: intt becomes int
-

3. Insert Characters

- Add a missing character to complete a valid token.
 - Example: flot → insert a → float
-

4. Replace Characters

- Replace a wrong character with the correct one.
 - Example: ni → replace n with i → if
-

5. Transpose Characters

- Swap nearby characters to form valid token.
 - Example: nt → swap → tn → might become int
-

Summary Table:

Recovery Action	Description
Panic Mode	Skip until a valid token is found
Delete Character	Remove invalid character
Insert Character	Add missing character
Replace Character	Change wrong character to correct one

Recovery Action	Description
-----------------	-------------

Transpose Characters	Swap adjacent characters
----------------------	--------------------------

Let me know if you want a **one-line short version** or a diagram for revision!

You said:

What are the limitations of static allocation?

ChatGPT said:

Here is a **simple and direct answer** for:

✅ **What are the limitations of Static Allocation?**

📌 **Static Allocation means memory is assigned at compile time, and locations are fixed during the program execution.**

❌ **Limitations of Static Allocation:**

1. No Flexibility

- Memory size must be known in advance.
- Cannot handle dynamic data like linked lists or variable-sized arrays.

2. Wastage of Memory

- If allocated memory is not fully used, it is wasted.
- If too little is allocated, program may fail.

3. No Support for Recursion

- Since local variables share the same memory, recursive calls can't maintain separate copies.

4. Fixed Lifespan of Variables

- Variables exist throughout the program, even when not needed.

5. Inefficient Use of Memory

- All memory is reserved whether or not it is used during runtime.

Example:

C

CopyEdit

```
int arr[1000]; // reserves memory at compile time, even if only 10 elements are used.
```

Let me know if you want a **short summary table** for revision!

You said:

) How is stack storage allocation strategy different from heap allocation strategy? Describe them and mentioning their merits and demerits.

ChatGPT said:

Here is a **simple and exam-ready answer** for:

 **How is Stack Storage Allocation Strategy different from Heap Allocation Strategy?**

Describe them with their merits and demerits.

1. Stack Allocation:

- Memory is allocated in a **Last In First Out (LIFO)** manner.
- Used for **local variables** and **function calls**.
- Memory is automatically **freed** when a function ends.
- Managed by the **compiler**.

Example:

c

CopyEdit

```
void func() {  
    int a = 5; // stack memory  
}
```

Merits of Stack Allocation:

- Fast allocation and deallocation.
- Simple to manage.
- Memory is automatically freed.

Demerits of Stack Allocation:

- Limited memory size (can cause **stack overflow**).
 - Cannot allocate variable-sized or long-lived data.
 - Not suitable for dynamic data structures.
-

2. Heap Allocation:

- Memory is allocated from the **heap** at **runtime**.
- Used for **dynamic memory**, such as arrays, linked lists, etc.
- Programmer must **manually allocate and free** memory.
- Managed by the **programmer**.

Example:

c

CopyEdit

```
int *p = (int*)malloc(sizeof(int)); // heap memory  
free(p);
```

Merits of Heap Allocation:

- Flexible and supports dynamic data.
- Suitable for data structures with variable size or unknown size.

Demerits of Heap Allocation:

- Slower than stack allocation.
 - Risk of **memory leaks** and **dangling pointers**.
 - Requires manual memory management.
-

Comparison Table:

Feature	Stack Allocation	Heap Allocation
---------	------------------	-----------------

Feature	Stack Allocation	Heap Allocation
Allocation Time	Compile-time / Runtime (LIFO)	Runtime
Speed	Faster	Slower
Memory Management	Automatic	Manual (malloc(), free())
Usage	Local variables, function calls	Dynamic memory, data structures
Lifespan	Short (till function ends)	Long (controlled by programmer)
Risk	Stack overflow	Memory leaks, dangling pointers

Let me know if you'd like this in **bullet format or diagram form** for your revision notes!

You said:

) Explain different polymorphic functions with suitable examples.

ChatGPT said:

Here's a **simple and exam-friendly answer** for:

 **Explain Different Polymorphic Functions with Suitable Examples**

 **What is Polymorphism?**

Polymorphism means "**many forms**".

It allows functions to behave **differently based on the input or context**.

Types of Polymorphic Functions:

1. Function Overloading (Compile-Time Polymorphism)

- Same function name with **different parameter types or numbers**.
- Decided **at compile time**.

Example in C++:

cpp

CopyEdit

```
int add(int a, int b) {  
    return a + b;  
}
```

```
float add(float a, float b) {  
    return a + b;  
}
```

 The function add() works for both int and float.

2. Function Overriding (Run-Time Polymorphism)

- Same function name in **base and derived class**.
- Decided **at runtime** using inheritance and virtual functions.

Example in C++:

cpp

CopyEdit

```
class Animal {  
public:  
    virtual void sound() {  
        cout << "Animal sound";  
    }  
};
```

```
class Dog : public Animal {  
public:  
    void sound() override {  
        cout << "Dog barks";  
    }  
};
```

👉 Function sound() behaves differently for Animal and Dog.

✅ Merits of Polymorphic Functions:

- Code reusability.
- Easy to maintain and extend.
- Supports flexible programming.

Comparison Table:

Type	Based On	When Decided	Example
Function Overloading	Arguments	Compile Time	add(int, int)
Function Overriding	Inheritance	Run Time	sound() in class

Let me know if you want a **one-line definition sheet** or short notes for revision!

You said:

. Explain in brief about equivalence of type expression

ChatGPT said:

Here is a **simple and brief answer** for:

Equivalence of Type Expressions

What is a Type Expression?

A **type expression** represents the type of a variable or expression in a programming language (e.g., int, float, pointer to int, array of float).

Equivalence of Type Expressions:

Two type expressions are said to be **equivalent** if they **represent the same type**.

There are **two main types of equivalence**:

1. Name Equivalence

- Types are equivalent **only if** they have **the same name**.
- Even if structure is same, different names mean not equivalent.

Example:

C

CopyEdit

```
typedef int marks;
```

```
typedef int score;
```

👉 marks and score are **not equivalent** under name equivalence.

2. Structural Equivalence

- Types are equivalent if they have **the same structure**, regardless of name.
- Looks at the **internal format**.

Example:

C

CopyEdit

```
typedef int A[10];
```

```
typedef int B[10];
```

👉 A and B are **equivalent** under structural equivalence.

 **Summary Table:**

Equivalence Type	Condition	Example Result
Name Equivalence	Same name	typedef int A; vs A = 
Structural Equivalence	Same structure/type format	int[10] vs int[10] = 

Let me know if you'd like **examples in diagrams or flowcharts** for quick revision!

You said:

Explain the various errors encountered in different phases of compiler.

ChatGPT said:

Here is a **simple and exam-friendly answer** for:

Errors Encountered in Different Phases of a Compiler

A compiler checks for different types of errors in **each phase** of compilation.
Here's a breakdown:

1. Lexical Errors (Lexical Analysis Phase)

- Caused by **invalid tokens** or characters.
- Detected by **lexical analyzer**.

Examples:

- Illegal characters (@, #)
 - Misspelled keywords (innt instead of int)
-

2. Syntax Errors (Syntax Analysis Phase)

- Occur when the **grammar rules** of the language are broken.
- Detected by the **parser**.

Examples:

- Missing semicolon (;)
 - Unmatched parentheses (if(x > 5)
 - Incorrect statement structure
-

3. Semantic Errors (Semantic Analysis Phase)

- Occur when statements are **grammatically correct** but **meaningless**.
- Detected during **semantic checking**.

Examples:

- Using an undeclared variable
 - Type mismatch (e.g., int x = "hello";)
 - Function return type mismatch
-

4. Logical Errors (Not Compiler Errors)

- The program **compiles and runs**, but gives **wrong output**.
- These are **not caught by the compiler**.

Examples:

- Wrong formula or condition
- Incorrect loop logic

5. Runtime Errors (During Program Execution)

- Detected when the program is **running**.

Examples:

- Division by zero
 - Accessing invalid memory
 - Array index out of bounds
-

Summary Table:

Phase	Error Type	Example
Lexical Analysis	Lexical Error	Invalid token: @var = 5;
Syntax Analysis	Syntax Error	Missing ; or)
Semantic Analysis	Semantic Error	int x = "abc";
Execution (Runtime)	Runtime Error	int a = 5/0;
Logic (Program Code)	Logical Error	Wrong formula: $\text{area} = 2 * \pi * r$

Let me know if you need a **diagram showing the compiler phases with errors** for easy understanding!

You said:

Explain the symbol table management system.

ChatGPT said:

Here is a **simple and exam-focused answer** for:

Symbol Table Management System

What is a Symbol Table?

A **symbol table** is a **data structure** used by a compiler to **store information about identifiers (names)** used in the source program.

It keeps track of:

- Variables
 - Functions
 - Objects
 - Classes
 - Labels, etc.
-

Information Stored in Symbol Table:

For each identifier, the symbol table stores:

- Name of the identifier
- Data type (int, float, etc.)
- Scope (local/global)
- Memory location or address
- Size and dimensions (for arrays)
- Type of symbol (variable/function/class)

 Example:

Identifier Type Scope Address

x int Local 1000

sum() func Global 2000

 Operations on Symbol Table:

1. **Insert(name, type, scope)** – Add a new entry
 2. **Lookup(name)** – Search for an identifier
 3. **Delete(name)** – Remove an entry (e.g., on scope exit)
 4. **Update(name, new_info)** – Modify existing data
-

 Implementation Methods:

- **Linear List** – Simple but slow
 - **Hash Table** – Fast lookup using hashing
 - **Binary Search Tree (BST)** – Faster search with ordered data
 - **Trie** – Efficient for string-based identifiers
-

 Importance of Symbol Table:

- Helps in **semantic analysis**
- Used for **type checking**

- Supports **scope management**
 - Helps during **code generation**
-

Summary Table:

Feature	Description
Purpose	Store identifier details
Key Operations	Insert, Lookup, Delete, Update
Implementations	Hash table, Linear list, BST, Trie
Usage in Compiler	Semantic analysis, type checking, etc.

Let me know if you want a **diagram or flowchart** of how the symbol table works!

You said:

Explain in detail different dynamic storage allocation strategies.

ChatGPT said:

Here is a **clear and simple explanation** for:

Different Dynamic Storage Allocation Strategies

What is Dynamic Storage Allocation?

It refers to **allocating memory at runtime**, based on the program's needs.
Used for data structures like **linked lists, trees, stacks, queues**, etc.

Main Strategies for Dynamic Storage Allocation:

1. First Fit

- Allocates the **first free block** that is **large enough**.
- Fast and simple.

Example:

Free blocks: 100B, 200B, 300B

Request: 150B → Allocated to 200B block

Advantages:

- Simple and fast
- Minimizes search time

Disadvantages:

- May cause **external fragmentation**
-

2. Best Fit

- Allocates the **smallest free block** that is **just enough**.
- Searches entire memory.

Example:

Free blocks: 100B, 200B, 300B

Request: 150B → Allocated to 200B block

Advantages:

- Less memory wastage

✗ Disadvantages:

- Slower due to full search
 - Creates many small unusable holes
-

3. Worst Fit

- Allocates the **largest available block**.
- Leaves large leftover space for future use.

Example:

Free blocks: 100B, 200B, 300B

Request: 150B → Allocated to 300B block

✓ Advantages:

- Reduces creation of small fragments

✗ Disadvantages:

- Wastes large memory blocks
 - Poor space utilization
-

4. Buddy System

- Memory is divided into blocks of size **power of 2**.
- Tries to reduce fragmentation.
- If a large block is needed, **buddies are merged**.

✓ Advantages:

- Balanced between speed and memory usage

- Easy to manage merging and splitting

✖ Disadvantages:

- May still cause **internal fragmentation**

🧠 Summary Table:

Strategy	How it works	Advantage	Disadvantage
First Fit	First block large enough	Fast, simple	External fragmentation
Best Fit	Smallest block that fits	Less memory waste	Slow search, tiny holes
Worst Fit	Largest block	Leaves large free blocks	May waste big blocks
Buddy System	Blocks split/merged in powers of 2	Efficient, manageable	Internal fragmentation

Let me know if you'd like a **diagram or animation-style steps** for any of these!

You said:

) What are the features and capabilities of the symbol table?

ChatGPT said:

Here is a **simple and exam-focused answer** for:

✅ Features and Capabilities of the Symbol Table

What is a Symbol Table?

A **symbol table** is a **data structure** used by the compiler to **store and manage information about identifiers** (like variables, functions, objects) in the source program.

Key Features of Symbol Table:

1. **Stores Identifier Information**
 - Name, type, scope, memory location, etc.
2. **Fast Lookup and Insertion**
 - Quickly finds and adds identifiers (especially using **hash tables**).
3. **Supports Scoping**
 - Handles **global and local scopes**, nested blocks, functions, etc.
4. **Error Checking**
 - Helps detect **multiple declarations, undeclared variables**, etc.
5. **Efficient Memory Management**
 - Helps manage variable addresses and storage during code generation.

Capabilities of Symbol Table:

Capability	Description
Insert (Add Entry)	Add new identifier and its attributes

Capability	Description
Lookup (Search Entry)	Find information of an identifier
Delete Entry	Remove identifiers when scope ends
Update Entry	Modify details like memory address, data type, etc.
Scope Handling	Maintain different scopes using a stack-based structure
Type Checking	Ensure operations are done between compatible types

Example Stored Data:

Name Type Scope Address

x int Local 1000

sum() func Global 2000

Benefits:

- Simplifies compiler design
 - Helps in semantic analysis
 - Supports efficient code generation and optimization
-

Let me know if you'd like a **symbol table working diagram or example** from a C/C++ program!

You said:

Explain implicit type conversion and explicit type conversion with example.

ChatGPT said:

Here's a **clear and simple answer** for:

✅ Implicit and Explicit Type Conversion (with Examples)

🔄 1. Implicit Type Conversion (Type Promotion)

- Done **automatically by the compiler**.
 - Converts a **smaller or lower type** to a **larger or higher type**.
 - Also called **type coercion**.
-

Example:

c

CopyEdit

```
int a = 5;
```

```
float b = a; // int automatically converted to float
```

➡ Here, a (int) is **implicitly converted** to float.

✅ Features:

- Automatic
 - No data loss (in most cases)
 - Follows **type hierarchy**: char → int → float → double
-

Common Situations:

- Mixing int and float in expressions:

c

CopyEdit

```
int x = 5;
```

```
float y = 2.5;
```

```
float z = x + y; // int x is promoted to float
```

2. Explicit Type Conversion (Type Casting)

- Done **manually by the programmer**.
 - Also called **type casting**.
 - Syntax: (type) expression
-

Example:

c

CopyEdit

```
float a = 5.9;
```

```
int b = (int) a; // float manually converted to int
```

 Here, a is **explicitly converted** to int (decimal part lost).

Features:

- Programmer-controlled

- May cause **data loss**
 - Useful when precision or format control is needed
-

Summary Table:

Feature	Implicit Conversion	Explicit Conversion
Who does it?	Compiler	Programmer
Also called	Type promotion/coercion	Type casting
Syntax	Automatic	(type) variable
Data Loss Risk	Low	Possible
Example	<code>int x = 5; float y = x;</code>	<code>float a = 5.5; int b = (int)a;</code>

Let me know if you'd like **MCQ practice** or **code snippets** to test your understanding!

You said:

What is activation tree? How control stack is maintained with the help of activation tree? Explain with example.

ChatGPT said:

Here is a **simple and exam-ready explanation** for:

What is an Activation Tree?

An **activation tree** is a **diagram** that represents **function calls and their relationships** during program execution.

Each **node** in the tree shows a **function (procedure) activation**, and edges show which function **calls which**.

Purpose of Activation Tree

- Helps to visualize **function calling sequence**
 - Shows how **functions are nested**
 - Represents **execution order** and **scope**
-

Basic Terms:

- **Activation**: A single execution of a function.
 - **Root**: The first (main) function.
 - **Child Node**: A function called by another function.
-

Example:

c

CopyEdit

```
void A() {
```

```
    B();
```

```
    C();
```

```
}
```

```
void B() {
```

```
D();  
}
```

```
void C() {}
```

```
void D() {}
```

```
int main() {  
    A();  
}
```

🌳 **Activation Tree for the above code:**

less

CopyEdit

main

|

A

/ \

B C

|

D

- main calls A

- A calls B and then C
 - B calls D
-

How Control Stack is Maintained Using Activation Tree?

A **control stack** keeps track of **active functions** (those that haven't returned yet).

Whenever a function is called:

- A **stack frame** is **pushed** onto the control stack.
When the function finishes:
 - The frame is **popped** from the stack.
-

Control Stack (Step-by-Step):

At runtime, for the above code:

1. main() is called → push(main)
2. main calls A() → push(A)
3. A calls B() → push(B)
4. B calls D() → push(D)
5. D finishes → pop(D)
6. B finishes → pop(B)
7. A calls C() → push(C)
8. C finishes → pop(C)
9. A finishes → pop(A)
10. main finishes → pop(main)

✓ **Key Points:**

- **Top of the stack = currently executing function**
 - When a function **calls another**, a new **activation record** is pushed
 - When it **returns**, that record is popped
-

 Summary Table:

Concept	Description
Activation Tree	Shows all function calls in tree form
Control Stack	Tracks active function calls (LIFO)
Push Operation	When a function is called
Pop Operation	When a function returns

Let me know if you'd like a **diagram image** of the activation tree and control stack!

You said:

Explain the concept of static, stack and heap allocation

ChatGPT said:

Here is a **simple and clear explanation** of:

✓ **Static, Stack, and Heap Allocation**

These are **memory allocation strategies** used to store variables during program execution.

1. Static Allocation

- **Memory is allocated at compile time**
- Size and location are **fixed**
- Used for **global variables, static local variables, and constants**

Example:

C

CopyEdit

```
int x = 10; // Static allocation
```

Advantages:

- Fast access
- No runtime overhead

Disadvantages:

- Fixed size
 - Cannot be changed during execution
-

2. Stack Allocation

- Memory is allocated in **Last-In-First-Out (LIFO)** order
- Used for **local variables, function parameters, and return addresses**

Example:

C

CopyEdit

```
void func() {  
    int a = 5; // Stack allocation  
}
```

- When func() is called, a is **pushed** onto the stack
- When func() ends, a is **popped**

✅ Advantages:

- Fast allocation/deallocation
- Automatically managed

❌ Disadvantages:

- Limited size (stack overflow)
- Lifespan is only during function execution

📌 3. Heap Allocation

- Memory is allocated **dynamically at runtime**
- Used for **objects, arrays, dynamic data structures** like linked lists

🧠 Example:

C

CopyEdit

```
int* ptr = (int*)malloc(sizeof(int)); // Heap allocation in C
```

✓ Advantages:

- Flexible size
- Data can persist until **manually freed**

✗ Disadvantages:

- Slower than stack
- Needs **manual deallocation** (risk of memory leaks)

Summary Table:

Allocation Type	When Allocated	Lifespan	Example Use	Pros	Cons
Static	Compile time	Entire program	Global/static variables	Fast, simple	Inflexible, fixed size
Stack	Runtime (function call)	During function	Local variables, args	Fast, auto cleanup	Limited size
Heap	Runtime (manual)	Until freed	Dynamic arrays, objects	Flexible size	Slower, memory leaks

Let me know if you want a **diagram of memory layout** showing stack, heap, and static areas!